Does the source exist? + Does the source actually support the claim?

AI Generation -> Reference Inventory -> Systematic Sampling ->
Prediction -> Authenticity Verification -> Claim-Support Verification
-> Quantitative Scores -> Submission

Failure Modes in Chained Tool-Calling Agents Operating Without Human Checkpoint

Abstract

Large language model (LLM)-based agents are increasingly being designed to perform tasks through repeated interactions with external tools such as search engines, databases, APIs, code interpreters, browsers, file systems, and enterprise services. Unlike conventional language-model applications that primarily generate text, tool-calling agents can transform model outputs into actions that modify external state. When several tool calls are chained without human checkpoints, a small error in planning, tool selection, parameter generation, interpretation of tool output, or state tracking can propagate through subsequent steps and produce increasingly severe failures. This paper examines the major failure modes associated with autonomous chained tool calling, focusing on error propagation, incorrect tool selection, parameter and schema errors, stale or misleading tool outputs, planning failures, state inconsistency, prompt injection, privilege escalation, irreversible actions, and inadequate recovery. Research on ReAct, Toolformer, API-Bank, Gorilla, AgentBench, InjecAgent, AgentDojo, ToolEmu, and Agent Security Bench demonstrates that tool use introduces capabilities as well as new reliability and security risks. The paper argues that chained execution changes the risk profile of an agent because downstream actions may treat an earlier model-generated assumption as an established fact. Human checkpoints therefore represent more than simple approval steps: they can function as control boundaries that interrupt error propagation and prevent uncertain plans from becoming irreversible actions. Current research increasingly emphasizes sandboxing, capability restriction, runtime monitoring, structured tool interfaces, adversarial evaluation, and dynamic benchmarks. However, significant gaps remain in measuring cumulative failure probability across long tool chains and in determining when an agent should autonomously continue, recover, request clarification, or stop. Future research should therefore combine reliability engineering, security controls, provenance tracking, uncertainty estimation, and adaptive human oversight into unified frameworks for safe agentic execution.

Keywords: large language models, AI agents, tool calling, function calling, autonomous agents, failure propagation, prompt injection, human oversight, agent security, reliability

1. Introduction

Large language models have evolved from systems that primarily generate natural-language responses into systems capable of interacting with external environments. Tool-augmented agents can retrieve information, execute calculations, manipulate files, browse websites, call APIs, write and execute code, and perform multi-step tasks. ReAct demonstrated an influential approach in which reasoning and acting are interleaved, allowing an LLM to use observations from an external environment to update its subsequent actions (Yao et al., 2023). Toolformer similarly demonstrated that language models can learn when to invoke APIs, what arguments to provide, and how to incorporate tool outputs into subsequent generation (Schick et al., 2023).

The increasing autonomy of these systems creates an important distinction between **single tool use** and **chained tool use**. In a single interaction, an incorrect tool call may produce one incorrect result. In a chain, however, the output of tool (T_1) can become an input to (T_2), whose output becomes an input to (T_3), and so on. The resulting process can be represented as:

[
S_0 \rightarrow A_1 \rightarrow O_1 \rightarrow A_2 \rightarrow O_2
\rightarrow \cdots \rightarrow A_n \rightarrow O_n]

where (S_0) is the initial state, (A_i) is an agent action or tool call, and (O_i) is the resulting observation. If an early action is inappropriate or its output is misinterpreted, later actions may operate on a corrupted state or incorrect assumption.

This problem becomes particularly significant when there are **no human checkpoints** between tool calls. A human checkpoint is a deliberate point at which a person reviews a proposed action, verifies an important intermediate result, approves a high-impact operation, or resolves ambiguity before execution continues. Without such checkpoints, the agent may autonomously transform uncertain reasoning into real-world actions.Existing benchmarks already demonstrate weaknesses in tool use, planning, and agent security.
API-Bank evaluates planning, API retrieval, and API calling and reports substantial remaining challenges in tool utilization. AgentBench evaluates agents across multiple interactive environments and identifies long-term reasoning, decision-making, and instruction-following as major obstacles.

This paper examines the failure modes that arise when tool calls are chained together without intermediate human validation. The central argument is that the principal risk is not merely that an individual tool call can fail. Rather, **errors can become inputs to subsequent decisions, allowing local failures to propagate into compound failures**.

2. Background and Related Work

2.1 Tool-Augmented Language Models

Tool use provides language models with capabilities that are difficult to obtain through language generation alone. Toolformer showed that external APIs can provide calculators, search systems, translation services, and other specialized capabilities.

Gorilla further investigated API invocation and highlighted problems including inaccurate arguments and hallucinated API usage. Its evaluation emphasized the importance of accurate API selection and parameter generation, especially when the number of available APIs is large or documentation changes over time (Patil et al., 2023).

A later survey by Qu et al. organizes tool learning into four broad stages: task planning, tool selection, tool calling, and response generation. This decomposition is useful for understanding chained failures because an error at any stage can affect every later stage.

2.2 Agent Evaluation

API-Bank provides a structured benchmark for tool-augmented LLMs, including 73 runnable APIs and hundreds of annotated tool-use dialogues. Its analysis shows that tool utilization involves multiple distinct competencies rather than a single capability.

AgentBench extends evaluation beyond isolated tool calls by evaluating LLMs as agents in multiple interactive environments. The study reports that poor long-term reasoning, decision-making, and instruction following remain important limitations.

These results are directly relevant to chained execution. A chain requires the model to maintain a coherent objective across multiple decisions rather than merely produce one valid API call.

2.3 Security of Tool-Using Agents

Tool use also creates security vulnerabilities because tool outputs may contain information that the model interprets as instructions. InjecAgent specifically studies indirect prompt injection in tool-integrated LLM agents and reports that external content can manipulate agents into harmful actions or private-data exfiltration.

AgentDojo provides a dynamic environment for evaluating prompt-injection attacks and defenses against agents operating on untrusted data. It includes realistic tasks involving email, banking, and travel-booking environments and demonstrates that modern agents can fail even without adversarial attacks, while prompt injections can further compromise security properties.

Agent Security Bench (ASB) expands security evaluation across multiple attack and defense categories and reports vulnerabilities involving system prompts, user prompts, tool use, and memory retrieval. Its results reinforce the observation that agent security cannot be treated as a problem confined to the underlying language model.

3. Main Failure Modes in Chained Tool Calling

3.1 Planning and Decomposition Failure

A chained agent must transform a high-level objective into a sequence of executable operations. Planning can fail when the model chooses an unnecessary step, omits a required dependency, misunderstands the order of operations, or commits prematurely to an incorrect strategy.
For example, suppose an agent is asked to analyze financial records and prepare a report. It might first retrieve transactions, then classify them, then calculate totals, then generate a report. If the classification procedure is wrong, subsequent calculations can be internally consistent while still being based on an incorrect interpretation.

The danger is therefore **structural consistency without semantic correctness**. Later steps may appear successful because they correctly process the outputs of earlier steps even though those outputs were wrong.

3.2 Tool-Selection Failure

An agent may select the wrong tool even when an appropriate tool exists. Tool-selection failure can result from ambiguous descriptions, overlapping capabilities, poor retrieval of tool documentation, or an incorrect interpretation of the user's objective.

API-Bank's explicit separation of planning, retrieving, and calling demonstrates that selecting and using tools are distinct competencies.

In a chained setting, wrong tool selection can be particularly damaging because the output may still look plausible. A search tool, database query, or estimation API may return a valid response even though it was not the appropriate source for the intended task.

3.3 Parameter and Schema Errors

Even when the correct tool is selected, the model must generate valid parameters. Errors may involve incorrect field names, invalid values, wrong units, omitted constraints, or inappropriate defaults.

Gorilla explicitly identifies accurate input-argument generation as a major challenge in API calling.

A parameter error can also become a propagation problem. Consider a chain:

[
T_1(x) \rightarrow y
]

[
T_2(y) \rightarrow z
]

If (x) is incorrectly specified, (y) may be wrong but syntactically valid. The second tool may therefore accept (y), producing (z) without raising an error. The chain has then converted a detectable local mistake into a less detectable downstream error.

3.4 Error Propagation

Error propagation is the defining reliability problem of chained execution.

Suppose each independent tool call has probability (p) of being correct. Under a simplified independence assumption, the probability that all (n) calls are correct is:

$$
P(\text{all correct}) = p^n
$$

Thus, if every step has a 95% probability of being correct, ten independent steps would have an all-correct probability of approximately:

$$
0.95^{10} \approx 0.599
$$

This calculation is only illustrative because real agent failures are neither independent nor identically distributed. Nevertheless, it demonstrates why long chains can be substantially less reliable than individual operations.
More importantly, errors can be **dependent**. An early incorrect assumption may systematically influence several subsequent decisions, causing correlated failures rather than independent ones. This means that simply multiplying per-step success rates may underestimate real-world risk.

3.5 Misinterpretation of Tool Outputs

Tool outputs are not automatically equivalent to verified facts. A tool can return incomplete, stale, ambiguous, malformed, or context-dependent information. The agent must interpret the output before using it in subsequent actions.

A particularly dangerous pattern occurs when the agent treats a tool response as authoritative even though the tool only returned a partial result. The resulting assumption can then become embedded in the next tool call.

This problem is related to the broader distinction between **data and instructions**. In tool-integrated systems, external content may contain text that the model interprets as an instruction rather than merely as data. AgentDojo and InjecAgent demonstrate why this distinction is difficult for current agents.

3.6 State-Tracking and Context Drift

Long chains require the agent to maintain an accurate representation of:

- the original user objective;
- completed actions;
- current external state;
- assumptions made during execution;
- information obtained from tools;
- constraints and permissions; and
- outstanding subgoals.

As the chain becomes longer, the possibility of state inconsistency increases. An agent may forget an earlier constraint, confuse an intermediate value with a final value, or continue executing after an external state has changed.

Agent evaluation research identifies long-term reasoning and decision-making as important obstacles for usable agents, making state management a central concern for autonomous chains.

3.7 Prompt Injection and Instruction Contamination

One of the most serious failure modes occurs when untrusted tool output contains

instructions designed to manipulate the agent. For example:

1. The user asks the agent to summarize documents.

2. The agent calls a document-retrieval tool.
3. One retrieved document contains malicious instructions.
4. The model interprets those instructions as part of its operational context.
5. The agent calls another tool to disclose information or perform an unauthorized action.

The important characteristic is that the attacker may not need direct access to the agent's system prompt. The malicious content can enter through a legitimate tool.

InjecAgent found substantial vulnerability to indirect prompt injection in tool-integrated agents, while AgentDojo provides a broader environment for evaluating this class of attack.

3.8 Privilege and Capability Escalation

Chained agents can combine individually harmless capabilities into a powerful sequence. For example, one tool may read files, another may send messages, and a third may access an external service. None of these capabilities necessarily creates catastrophic risk by itself. However, chaining them can produce a path from information retrieval to unauthorized disclosure or external action.

This suggests that security should evaluate not only **individual tool permissions** but also **compositions of permissions**.

ASB's evaluation of attacks across system prompts, tool usage, and memory retrieval illustrates why agent security must consider the entire workflow rather than only isolated model responses.

3.9 Irreversible and High-Impact Actions

A major distinction between informational and operational tools is reversibility.

Examples of relatively reversible actions include:

- retrieving public information;
- calculating a value;
- generating a draft.

Examples of potentially irreversible or high-impact actions include:

- deleting files;
- transferring funds;
- sending messages;
- changing account permissions;
- modifying production infrastructure;
- publishing information.

An autonomous chain that proceeds directly from reasoning to irreversible execution has a substantially different risk profile from one that pauses before high-impact operations.

3.10 Recovery Failure

Agents do not only need to perform actions correctly; they need to recover when actions fail.

A weak recovery policy may:

1. repeat the same failed call;
2. modify parameters without understanding the cause;
3. switch tools unnecessarily;
4. conceal the failure in the final answer; or
5. continue executing despite an uncertain state.

A robust agent should instead distinguish between recoverable and non-recoverable failures and know when to stop.

This is especially important because automatic retry mechanisms can amplify rather than reduce risk. Repeating a failed read operation may be harmless, whereas repeatedly attempting a state-changing operation may produce multiple unwanted effects.

4. Why the Absence of Human Checkpoints Matters

Human checkpoints can be understood as **control barriers** in the agent's execution pipeline. Without a checkpoint:

$$
[
\text{Plan} \rightarrow \text{Tool}_1 \rightarrow \text{Tool}_2 \rightarrow \text{Tool}_3 \rightarrow \text{Action}
]
$$

With checkpoints:

$$
[
\text{Plan} \rightarrow \text{Tool}_1 \rightarrow \boxed{\text{Review}} \rightarrow \text{Tool}_2 \rightarrow \boxed{\text{Review}} \rightarrow \text{Action}
]
$$

The checkpoint provides an opportunity to detect incorrect assumptions before they propagate.

However, human oversight is not necessarily required after every operation. Excessive intervention can reduce efficiency and undermine the purpose of autonomous agents. A more useful model is **risk-adaptive oversight**, in which checkpoints are triggered by properties such as:

- high uncertainty;
- sensitive data;
- unusual tool combinations;
- irreversible actions;
- large financial or operational consequences;
- privilege changes;
- conflicts between tool outputs;
- anomalous behavior;
- deviations from the original plan.

This approach treats human oversight as a scarce safety resource rather than as a universal requirement.

5. Current Approaches and Developments

5.1 Structured Tool Interfaces

Structured APIs and explicit schemas can reduce parameter-generation errors by constraining the model's output space. API-Bank and related tool-learning research demonstrate the importance of evaluating tool selection and calling as separate capabilities.

5.2 Retrieval-Augmented Tool Documentation

Gorilla demonstrates that retrieving current API documentation can improve the reliability of API calls and reduce problems caused by changing tool interfaces.

5.3 Sandboxing and Simulation

ToolEmu proposes using an LM-emulated sandbox to identify risks associated with tool-using agents without requiring every real-world tool to be implemented. This provides a scalable approach to testing potentially dangerous actions before deployment.

5.4 Adversarial Evaluation

AgentDojo and InjecAgent demonstrate the importance of testing agents against indirect prompt injection and untrusted external content rather than evaluating only benign tasks.

ASB similarly evaluates a broad range of attacks and defenses across different agent scenarios, emphasizing that agent security involves multiple layers.

5.5 Runtime Monitoring

Runtime monitoring can inspect tool calls before execution, compare actions with policy, detect anomalous sequences, and prevent unauthorized operations. Such mechanisms are especially important for chains because the safety of an individual action may depend on what happened earlier.

Recent research increasingly emphasizes runtime controls and capability restrictions rather than relying solely on natural-language instructions to the model.

6. Research Challenges and Limitations

A major limitation of current research is that many evaluations focus on task completion rather than the detailed reliability of the execution process. A system can achieve a successful final answer while making unsafe intermediate

decisions, particularly when the benchmark does not expose those decisions as separate evaluation targets.

Recent evaluation surveys identify a gap between benchmark performance and real deployment requirements, including safety, robustness, cost, maintainability, and workflow integration.

Another challenge is the absence of standardized metrics for **cumulative chain reliability**. Existing metrics often report task success, tool-call accuracy, attack success, or similar measures separately. What is needed is a framework that records where in a chain an error occurred, whether it propagated, whether recovery was attempted, and whether a checkpoint could have prevented the final failure.
Security evaluation also remains difficult because attacks and defenses evolve rapidly. Agent Security Bench and AgentDojo demonstrate that adversarial evaluation must account for multiple attack surfaces and adaptive behavior.

Finally, human oversight itself requires careful design. A poorly designed checkpoint may merely ask a human to approve an action without providing enough context to understand the decision. Effective oversight therefore requires concise explanations, relevant evidence, action previews, and clear indications of uncertainty.

7. Research Gaps and Future Directions

7.1 Chain-Level Reliability Metrics

Future benchmarks should report not only whether an agent completed a task but also:

- number of tool calls;
- number of retries;
- first point of failure;
- error propagation length;
- recovery success;
- irreversible actions attempted;
- policy violations;
- human interventions required.

This would allow researchers to distinguish a short successful trajectory from a fragile trajectory that happens to reach the correct result.

7.2 Provenance-Aware State Management

Agents should maintain explicit provenance for

important intermediate values. Instead of

storing only: “Customer balance = X”

the system should retain:

“Customer balance = X, obtained from Tool A at time T, using

query Q, with confidence/validation status V.” Such provenance would

make it easier to identify stale or unverified assumptions before they enter

subsequent tool calls. **7.3 Adaptive Checkpoint Placement**

Rather than fixed checkpoints, future systems could estimate the risk of continuing autonomously. A checkpoint could be triggered when the predicted expected loss of autonomous continuation exceeds the cost of human review.

Conceptually:

$$
\text{Request human review if } P(\text{harm} \mid s,a)\times C_{\text{harm}} C_{\text{review}}
$$

where (s) is the current state, (a) is the proposed action,
(C_{harm}) is the estimated consequence of failure, and
(C_{review}) is the cost of human intervention.

This creates a principled basis for balancing autonomy and oversight.

7.4 Compositional Security Analysis

Security testing should examine combinations of tools rather than only individual tool permissions. A tool chain should be evaluated as a graph in which nodes represent capabilities and edges represent possible transitions between capabilities.

This could reveal dangerous compositions that are

invisible when every tool is evaluated independently. **7.5**

Failure-Aware Agent Architectures

Future agents should explicitly represent uncertainty and execution state. A robust agent should be able to classify an intermediate result as:

- verified;
- plausible but unverified;
- contradictory;
- stale;
- malformed;
- unsafe to act upon.

This would help prevent the common failure pattern in which an uncertain observation silently becomes a definitive premise for the next action.

8. Conclusion

Chained tool-calling agents represent an important transition from language generation toward autonomous interaction with external systems. ReAct and Toolformer established important foundations for reasoning and tool use, while later benchmarks such as API-Bank, AgentBench, AgentDojo, InjecAgent, ToolEmu, and Agent Security Bench have exposed limitations in planning, API invocation, long-horizon reasoning, security, and robustness.

The central problem with autonomous chains is not simply that an agent can make a mistake. It is that **an early mistake can become the basis for later decisions**. Incorrect plans, tool choices, parameters, interpretations, or state assumptions can

therefore propagate across multiple steps. When the chain includes high-impact or irreversible operations, the resulting failure can move from an informational error to a real-world consequence.

Human checkpoints provide an effective mechanism for interrupting this propagation, but continuous human approval is neither scalable nor desirable. The more promising direction is adaptive oversight combined with structured tool interfaces, capability restrictions, provenance tracking, runtime monitoring, sandboxing, adversarial testing, and explicit uncertainty management.

Ultimately, safe autonomous tool use requires systems to know not only **how to call a tool**, but also **when not to call it, when to verify its output, when to recover, and when to stop**. Future research should therefore evaluate agents as complete socio-technical execution systems rather than as language models that merely produce successful final answers.

References

1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR 2023). arXiv:2210.03629. 2. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. Advances in Neural Information Processing Systems 36 (NeurIPS 2023). DOI: 10.52202/075280-2997; arXiv:2302.04761.
3. Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2023). *Gorilla: Large Language Model Connected with Massive APIs*. arXiv:2305.15334.
4. Li, M., Zhao, Y., Yu, B., Song, F., Li, H., Yu, H., Li, Z., Huang, F., & Li, Y. (2023). *API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs*. Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 3102–3116. DOI: 10.18653/v1/2023.emnlp-main.187; arXiv:2304.08244.
5. Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., & Tang, J. (2023). *AgentBench: Evaluating LLMs as Agents*. arXiv:2308.03688.
6. Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., Xu, J., & Wen, J.-R. (2025). *Tool learning with large language models: a survey*. Frontiers of Computer Science, 19(8), 198343. DOI: 10.1007/s11704-024-40678-2.
7. Zhan, Q., Liang, Z., Ying, Z., & Kang, D. (2024). *InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents*. Findings of the Association for Computational Linguistics: ACL 2024, 10471–10506. DOI: 10.18653/v1/2024.findings-acl.624.
8. Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. Advances in Neural Information Processing Systems 38 (NeurIPS 2024), Datasets and Benchmarks Track. arXiv:2406.13352.
9. Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., & Hashimoto, T. (2024). *Identifying the Risks of LM Agents with an LM-Emulated Sandbox*. International Conference on Learning Representations (ICLR 2024).
10. Zhang, H., Huang, J., Mei, K., Yao, Y., Wang, Z., Zhan, C., Wang, H., & Zhang, Y. (2025). *Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-Based Agents*. International Conference on Learning Representations (ICLR 2025).
11. Yehudai, A., Eden, L., Li, A., Uziel, G.,

Zhao, Y., Bar-Haim, R., Cohan, A., & Shmueli-Scheuer, M. (2025). *Survey on Evaluation of LLM-based Agents*. arXiv:2503.16416.

12. Alpay, F., & Alpay, T. (2026). *AgentSecBench: Measuring Prompt Injection, Privacy Leakage, and Tool-Use Integrity in LLM Agents*. arXiv:2605.26269. DOI: 10.48550/arXiv.2605.26269.
13. Hofer, D., Debenedetti, E., & Tramèr, F. (2026). *Assessing Automated Prompt Injection Attacks in Agentic Environments*. arXiv:2606.10525.
14. *Securing LLM agents: From prompt sanitization to autonomous red teaming and beyond*. (2026). Internet of Things and Cyber-Physical Systems, 5, 185–209. DOI: 10.1016/j.iotcps.2026.03.001.
15. *From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows*. (2026). ICT Express, 12(2), 353–383. DOI: 10.1016/j.icte.2025.12.001.